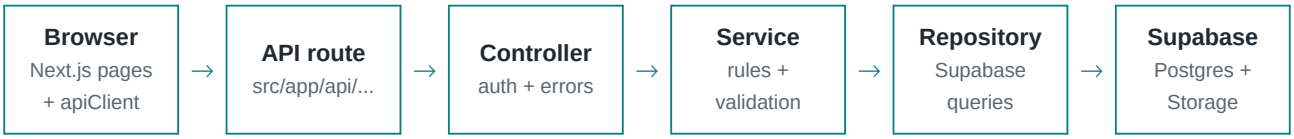
How YurCore Learning Works

Frontend, backend and database handling of the LMS · 27 October 2026

In one sentence. Admins build courses in the admin panel; practitioners (doctors) take them in the practitioner portal. Every action goes from a Next.js page to an API route on the YurCore server, which checks the login and role, applies the LMS rules, and reads or writes Supabase (Postgres for data, a private Storage bucket for files). Each YurCore-built course is also mirrored into Moodle.

1. The big picture

Every request follows the same path through the code. The browser never talks to the database directly.



Layer	Where	What it does
Pages	`src/app/admin/(app)/learning/` `src/app/(portal)/pro/learning/`	Screens for admins and practitioners. Client components that call the API.
API client	`_lib/api.ts` (admin and learner)	Small typed wrappers over `apiClient`: sends cookies, parses JSON, turns errors into messages.
API routes	`src/app/api/admin/learning/...` `src/app/api/learning/...`	Thin route files; each one calls one controller method.
Controller	`learning.controller.ts`	Reads the logged-in user, runs the service, maps errors to safe JSON, writes the audit log for admin actions.
Services	`learning-admin.service.ts` `learning.service.ts` `learning-moodle-sync.service.ts`	All the rules: who may do what, input validation (Zod), publish checks, progress, quiz grading, Moodle sync.
Repository	`learning.repo.ts`	Every Supabase query and Storage call, using the server-only service key.
Database	Supabase project	Postgres tables (RLS on) and the private Storage bucket learning.

2. Frontend

Admin side (course building)

Page	Purpose
`/admin/learning`	Course list with stat cards (total, drafts, published, learners), search, category and status filters, Build / Preview / Edit / Delete.
`/admin/learning/new`	Course details form (`CourseDetailsForm`): title, short and full description, category, level, duration, objectives, thumbnail.
`/admin/learning/[id]`	Course builder (`CourseBuilder`): sections, content items (`AddContentDialog`, `ContentEditor`, `QuizBuilder`, `FileDrop`), drag to reorder, preview, learners and responses, publish (`PublishDialog`).

Page	Purpose
`/admin/learning/[id]/settings`	Edit the course details later.

Practitioner side (learning)

Page	Purpose
`/pro/learning`	All Courses catalogue.
`/pro/learning/my-learning`	Courses the practitioner has started.
`/pro/learning/completed`	Courses the practitioner has finished.
`/pro/learning/courses/[id]`	Course player: sections, video / image / PDF / lesson viewer, quizzes, assignments, feedback, progress.

How the catalogue page loads courses (Show More)

1. `CourseList` asks the server for the first **12** courses for the current tab, search text and category (`LEARNING_PAGE_SIZE`).
2. The server answers with `{ items, nextCursor, hasMore, total }`. The page shows the cards and "Showing 12 of N courses".
3. **Show More Courses** sends the `nextCursor` back and appends the next 12. Repeat clicks are ignored while a batch loads; cards already shown are never added twice.
4. At the end, "You've reached the end of the courses" and **Show Less** appear. Show Less collapses back to the first 12 without calling the server.
5. Changing the search (after a 300 ms pause), category or tab cancels any request still running and starts again from the first batch.
6. The cards scroll inside their own panel, so the header, tabs, search and category filter stay visible.

Building blocks

- `CourseCard`: 16:9 thumbnail (or a neutral "No preview image" panel), category, status badge, title, description, duration, item count, level, progress bar (in-progress only), and Start course / Continue / Review course.
- `LearningTabs`: page header and the three tabs. `CategoryFilter`: custom keyboard-accessible category dropdown.
- **File uploads go straight to Storage.** The admin page asks the API for a signed upload link, then the browser uploads the file directly to Supabase Storage with a progress bar (`uploadFile`). Large videos never pass through the YurCore server.
- `apiClient` sends the login cookies with every call. On a 401 it clears the local session and sends the user to the login page.

3. Backend

Login and roles

- Login is by OTP. The server sets two HttpOnly cookies: `access_token` and `refresh_token` (JWTs signed with `JWT_SECRET`).
- Every learning route calls `requireAuth` to get the user from the cookie.
- **Admin and super_admin** may manage courses (every admin service method starts with `requireAdmin`).
- **Doctors / practitioners** with a practitioner profile may learn (`requirePractitioner`). Admins, assistants and patients are refused the learner routes.

API routes

Route	Who	What
`GET /api/learning/courses?scope&q&category&cursor&limit`	Practitioner	One catalogue batch (12 by default, at most 48).
`GET /api/learning/categories`	Practitioner	Categories of published courses.
`GET /api/learning/courses/[id]`	Practitioner	Course player data with signed file links and own progress.
`POST .../start`	Practitioner	Enroll in the course.
`POST .../content/[cid]/complete`	Practitioner	Mark an item done; updates course completion.
`POST .../content/[cid]/attempts`	Practitioner	Start a quiz attempt (questions without answers).
`POST .../attempts/[aid]`	Practitioner	Submit answers; the server grades and returns the review.
`POST .../content/[cid]/response`	Practitioner	Assignment text or feedback rating.
`GET|POST /api/admin/learning/courses`	Admin	List all courses / create a course.
`GET|PATCH|DELETE .../courses/[id]`	Admin	Course details / partial update / permanent delete.
`POST .../[id]/publish`	Admin	Publish or unpublish (with publish checks).
`POST|DELETE .../[id]/uploads`	Admin	Get a signed upload link / discard an upload.
`.../[id]/sections, /layout, /content`	Admin	Add, rename, delete, reorder sections and items; save quizzes.
`.../[id]/learners, .../responses`	Admin	Learner progress and submissions.
`.../[id]/sync, /api/admin/learning/sync`	Admin	Sync one course / all courses with Moodle.

Rules the services enforce

- **Validation:** every input is checked with Zod (lengths, levels, UUIDs, file types, exactly one correct answer per question). Update schemas have no default values, so a partial update (for example only a new thumbnail) never blanks other fields.
- **Publish checks:** a course needs at least one item; file items need their file; quizzes need questions.
- **Drafts are invisible to learners:** list, details, start, quiz and response calls all return 404 for unpublished courses.
- **Uploads:** the file must sit in that course's folder for its kind and exist with an allowed type before a row may point to it.
- **Quiz grading** happens on the server; correct answers are only sent back after submitting. Time limits get a 2-minute grace period; attempt limits are enforced.
- **Course completion** is re-checked each time the learner completes an item: it is set once every item is done, and cleared if the course has since gained items.
- **Errors:** internal details are logged on the server; the browser only gets a safe message.
- **Audit log:** every successful admin action is written to the audit log.

Catalogue pagination (keyset)

Courses are ordered by `published_at` (newest first, empty last) and then by `id`. The cursor is the sort key of the last card shown, encoded as opaque base64 and validated on every request. The next batch starts strictly after it, so a course published while someone is browsing can never duplicate or shift cards. The server reads

13 rows to know whether more exist. Only the 12 courses of the batch get item counts, the learner's progress, instructor names and signed thumbnail links.

Moodle integration

- Courses built in YurCore (`origin = yurcore`) are created in Moodle first, then mirrored there after every change (sections, files, lessons, quizzes, assignments).
- Courses created directly in Moodle (`origin = moodle`) are imported and kept up to date from Moodle; their content is edited in Moodle.
- Publishing shows the course in Moodle; unpublishing hides it. Deleting removes it from Moodle first, then from YurCore.
- Sync bookkeeping (`moodle_cmid`, `moodle_sync_hash`, `moodle_file_sig`) means a sync only sends what changed and does not re-upload large files.

4. Database (Supabase)

Tables and relationships

Table	Key columns	Belongs to
`meyveda_learning_courses`	title, short_description, description, category_name, level, duration_minutes, learning_objectives, thumbnail_path, status (draft/published), published_at, origin, moodle_course_id, created_by	users (creator)
`meyveda_learning_sections`	title, description, position, moodle_section_id	course
`meyveda_learning_content`	content_type (video, image, document, file, lesson, quiz, question, assignment, feedback), title, description, body, settings (JSON), storage_path, file_name, mime_type, size_bytes, position	section + course
`meyveda_learning_quizzes`	passing_score, time_limit_minutes, attempts_allowed	content item
`meyveda_learning_quiz_questions`	question, explanation, position	quiz
`meyveda_learning_quiz_options`	option_text, is_correct (at most one per question), position	question
`meyveda_learning_enrollments`	started_at, completed_at; unique per course + user	course + user
`meyveda_learning_progress`	completed_at; unique per item + user	content item + user
`meyveda_learning_quiz_attempts`	answers (JSON), correct_count, score_percent, passed	quiz + user
`meyveda_learning_responses`	response_text, rating; unique per item + user	content item + user
`moodle_user_mappings`	moodle_user_id, moodle_username	user

course → sections → content → quiz → questions → options. Deleting a course deletes everything below it (sections, content, quizzes, enrollments, progress, attempts, responses) through ON DELETE CASCADE, and the service removes its files from Storage.

Indexes that exist today

- courses: status · sections: (course_id, position) · content: (section_id, position), (course_id), (moodle_cmid)
- quiz questions: (quiz_id, position) · options: (question_id, position) · enrollments: (user_id) + unique (course_id, user_id)

- progress: (user_id, course_id) + unique (content_id, user_id) · attempts: (quiz_id, user_id)

File storage

- One **private** bucket, `learning`. Layout: `courses/<course-id>/{thumbnail, videos, images, documents, files}/<uuid>-<name>`.
- Database rows store only the path, file name, type and size, never the file itself.
- Admins upload with a signed upload link; learners and admins read with signed links valid for 3 hours.
- Allowed types: PNG/JPG/WebP/GIF images, MP4/WebM/MOV/OGG video, PDF/Word/PowerPoint, Excel, CSV, text, ZIP and audio. There is no app-level size limit; the Supabase project's upload limit applies (50 MB per file on the Free plan).

Security model

- Row Level Security is enabled on every learning table with no policies, so browser keys cannot read or write them at all.
- Only the YurCore server touches the data, with the service key that never leaves the server.
- Every query is scoped by the logged-in user's id for personal data (progress, attempts, responses, enrollments).
- Supabase returns at most 1,000 rows per request; the repository reads larger lists in batches so nothing is silently cut off.

5. Key flows, step by step

Admin creates and publishes a course

1. Fill in the course form and save. The server creates the Moodle course, then the YurCore course with one "Introduction" section.
2. If a thumbnail was chosen, the browser uploads it with a signed link and the course is updated with its path.
3. Add sections and items in the builder. File items upload first, then the item row is saved pointing to the file.
4. Click Publish. The server runs the publish checks, shows the course in Moodle, sets status = published and published_at.

Practitioner takes a course

1. Opens the catalogue: 12 published courses, with state Not started / In progress / Completed from their own records.
2. Clicks Start course: an enrollment row is created; the player loads sections with signed file links.
3. Each finished item adds a progress row. When all items are done, the enrollment gets completed_at.
4. Quizzes: start an attempt (questions only), submit, the server grades and stores the attempt and returns the review.
5. Assignments and feedback are saved as one response per item and user, visible to admins.

6. Known issues and recommendations

Security

- `JWT_SECRET` **is not set** in .env.local, so logins are signed with a fallback secret written in the source code. If production also lacks it, anyone with the code could forge an admin login. Set a strong secret (users will be logged out once).
- API routes accept the 7-day refresh token as a normal login, which weakens the 15-minute access-token expiry.

- **Admin search** matches titles only (in the browser); practitioner search also matches descriptions.
- **Admin course list** still loads every course at once (no pagination yet).
- **Indexes worth adding** (after checking query plans on a copy): courses (`status, published_at desc, id desc`) for the catalogue, and a `pg_trgm` text index for search at large scale.
- **Backups:** the Free plan has no downloadable or point-in-time backups, and Storage files are never in database backups. Export both regularly.
- **Test data:** 30 test courses, one draft ("QA UI Test"), Dr. Jaya's test progress and matching Moodle courses were created in this project during testing.